# EC-341 Digital Systems Design
# Experiment # 6

Analysis of Implementation of Combinational Circuits Using Different Approaches – Decoders and Multiplexers

## Objectives:

- Combinational Circuits
- Implementation using Discrete Gates
- Implementation using Decoders
- Implementation using Multiplexers
- Lab Tasks

## 1. Combinational Circuits

Combinational Circuits (CC) are circuits made up of different types of logic gates. A logic gate is a basic building block of any electronic circuit. The output of the combinational circuit depends on the values at the input at any given time. The circuits do not make use of any memory or storage device. Some of the most common types of combinational circuit include adder, subtractor, multiplexer and de-multiplexer.

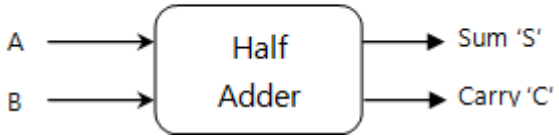
## 2. The Adders

An adder is a digital circuit that is used to perform the addition of numeric values. It is one of the most basic circuits and is found in arithmetic logic units of computing devices. There are two types of adders. Half adders compute single digit numbers, while full adders compute larger numbers.

### 2.1 Half Adder

The half adder adds two single digit binary numbers and forms the foundation for all addition operations in computing. If we have two single binary digits, A and B, then the half adder adds them with the circuit carrying two outputs, the sum and the carry. The carry represents any overflow from the addition of the two numbers. This is represented in the following block diagram figure:

Figure 1: Half Adder



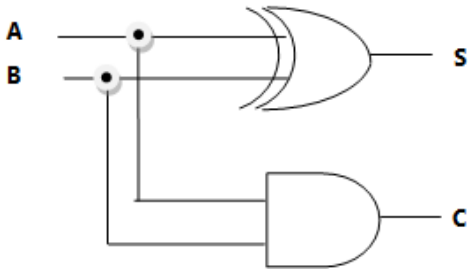

In addition, the following truth table demonstrates all the possible outputs for various input combinations of the half adder.

| A | B | S | C |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

Table 1: Truth Table - Half Adder

The next figure represents the logic circuit of the half adder:

Figure 2: Half Adder- Logic Circuit

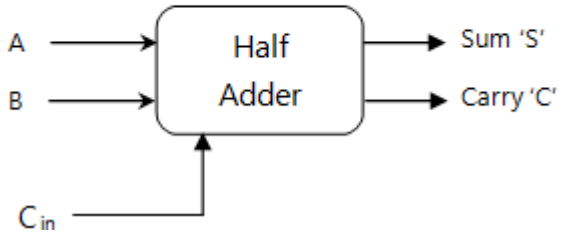



The sum S is represented by the Boolean Expression S = A'B + AB' and C = AB

## 2.2 Full Adder

The full adder overcomes the disadvantages of the half adder in that it can add two single bit numbers in addition to the carry digit at its input as seen in this figure:

Figure 3: Full Adder




The next truth table shown here demonstrates all the possible outputs for various input combinations with the carry input digit:

| A | B | Cin | Co | S |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |

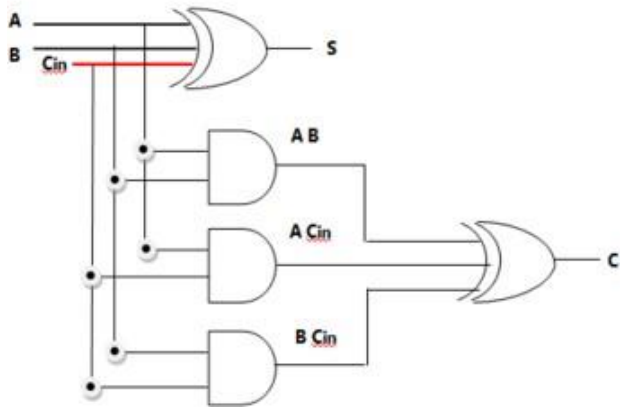
Boolean expression for the full adder is S = A'B'Cin + A'BCin' + AB'Cin' + ABCin and C = A'BCin + AB'Cin + ABCin' + ABCin. This is where A and B are all the possible binary inputs and C is the carry in. For example, if A is 0 and B is 0 and the Cin is 1, then:

*S = (0'0'1)+(0'01')+(00'1')+(001) = (111)+(100)+(010)+(001) = (1)+(0)+(0)+(0) = 1*

*C = (0'01)+(00'1)+(001')+(001) = (101)+(011)+(000)+(001) = (0)+(0)+(0)+(0) = 0*

*S = 1 and C = 0*

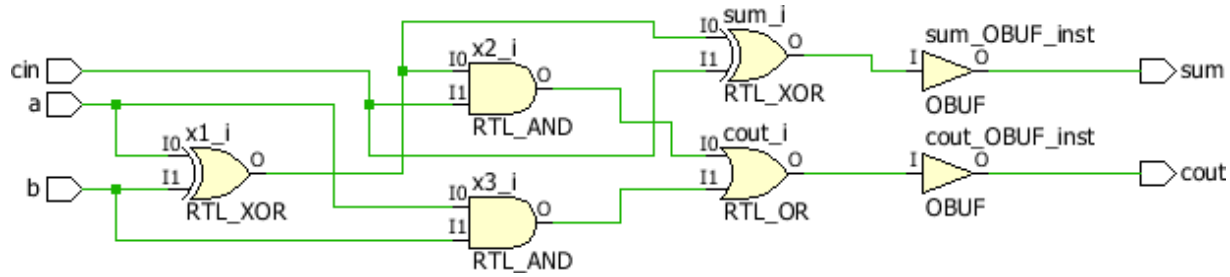
Figure 3: Full Adder- Logic Circuit




## Gate Level Implementation: -

```
module fulladder(
    input a,
    input b,
    input cin,
    output sum,
    output cout
    );
    wire x1,x2,x3;
    xor(x1,a,b);
    and(x3,a,b);
    xor(sum,x1,cin);
    and(x2,x1,cin);
    or(cout,x2,x3);
endmodule
```

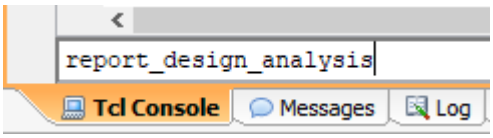
## Schematic: -




## Analysis: -

Analysis based on timing and complexity (Area Comparison).

## Timing: -




| Characteristics | Path #1 |
|---|---|
| Path Type | SETUP |
| Requirement | 0.000 |
| Path Delay | 5.377 |
| Logic Delay | 3.778(71%) |
| Net Delay | 1.600(29%) |
| Clock Skew | 0.000 |
| Slack | inf |
| Clock Relationship | Safely Timed |
| Timing Exception | |
| Logic Levels | 3 |
| Logical Path | IBUF LUT3 OBUF |
| Start Point Clock | input port clock |
| End Point Clock | |
| Start Point Pin Primitive | a |
| End Point Pin Primitive | sum |
| Start Point Pin | a |
| End Point Pin | sum |

| | |
|---|---|
| Comb DSP | 0 |
| DOA_REG | 0 |
| DOB_REG | 0 |
| MREG | 0 |
| PREG | 0 |
| BRAM Crossings | 0 |
| DSP Crossings | 0 |
| IO Crossings | 0 |
| Config Crossings | 0 |
| SLR Crossings | 0 |
| Bounding Box Size | 0% x 0% |
| Clock Region Distance | NA |
| PBlocks | 0 |
| High Fanout | 2 |
| Cumulative Fanout | 4 |
| Dont Touch | 0 |
| Mark Debug | 0 |
| Fixed Loc | 0 |
| Fixed Route | 0 |
| Hold Fix Detour | 0 |
| Combined LUT Pairs | 1 |

* Bounding box calculated as % of dimensions for the target device (156, 199)

Area: -

Report Cell Usage:

| | Cell | Count |
|---|---|---|
| 1 | LUT3 | 2 |
| 2 | IBUF | 3 |
| 3 | OBUF | 2 |

Report Instance Areas:

| | Instance | Module | Cells |
|---|---|---|---|
| 1 | top | | 7 |

Behavioral Level Implementation: -

```
module FA(
    input wire a,
    input wire b,
    input wire carry_in,
    output reg sum,
    output reg carry_out
    );

    always @(a or b or carry_in)
    begin
     if(a==0 && b==0 && carry_in==0)
      begin
       sum=0;
       carry_out=0;
      end

     else if(a==0 && b==0 && carry_in==1)
      begin
       sum=1;
       carry_out=0;
      end

     else if(a==0 && b==1 && carry_in==0)
      begin
       sum=1;
       carry_out=0;
      end

     else if(a==0 && b==1 && carry_in==1)
      begin
       sum=0;
       carry_out=1;
      end

     else if(a==1 && b==0 && carry_in==0)
      begin
       sum=1;
       carry_out=0;
      end

     else if(a==1 && b==0 && carry_in==1)
      begin
       sum=0;
       carry_out=1;
      end

     else if(a==1 && b==1 && carry_in==0)
      begin
       sum=0;
       carry_out=1;
      end

     else if(a==1 && b==1 && carry_in==1)
      begin
       sum=1;
       carry_out=1;
      end

    end
endmodule
```

Timing: -

| Characteristics | Path #1 |
|---|---|
| Path Type | SETUP |
| Requirement | 0.000 |
| Path Delay | 7.017 |
| Logic Delay | 4.641(67%) |
| Net Delay | 2.377(33%) |
| Clock Skew | 0.000 |
| Slack | inf |
| Clock Relationship | Safely Timed |
| Timing Exception | |
| Logic Levels | 4 |
| Logical Path | IBUF LUT3 LDCE OBUF |
| Start Point Clock | input port clock |
| End Point Clock | |

| Start Point Pin Primitive | carry_in |
|---|---|
| End Point Pin Primitive | carry_out |
| Start Point Pin | carry_in |
| End Point Pin | carry_out |
| Comb DSP | 0 |
| DOA_REG | 0 |
| DOB_REG | 0 |
| MREG | 0 |
| PREG | 0 |
| BRAM Crossings | 0 |
| DSP Crossings | 0 |
| IO Crossings | 0 |
| Config Crossings | 0 |
| SLR Crossings | 0 |
| Bounding Box Size | 0% x 0% |
| Clock Region Distance | NA |
| PBlocks | 0 |
| High Fanout | 6 |
| Cumulative Fanout | 9 |
| Dont Touch | 0 |
| Mark Debug | 0 |
| Fixed Loc | 0 |
| Fixed Route | 0 |
| Hold Fix Detour | 0 |
| Combined LUT Pairs | 0 |

* Bounding box calculated as % of dimensions for the target device (156, 199)

Area: -

Report Cell Usage:

| | Cell | Count |
|---|---|---|
| 1 | LUT3 | 6 |
| 2 | LDC | 2 |
| 3 | IBUF | 3 |
| 4 | OBUF | 2 |

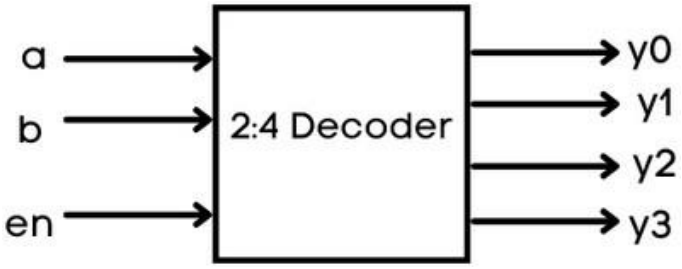
Report Instance Areas:

| | Instance | Module | Cells |
|---|---|---|---|
| 1 | top | | 13 |

## Decoders: -

A decoder is a combinational logic circuit that has 'n' input signal lines and 2n output lines. In the 2:4 decoder, we have 2 input lines and 4 output lines. In addition, we can provide "enable" to the input to ensure the decoder is functioning whenever enable is 1 and it is turned off when enable is 0 (which is optional). The truth table, logic diagram, and logic symbol are given below:

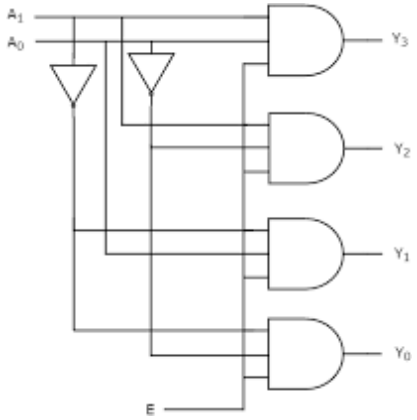
## Block Diagram: -




## Truth Table: -

| Inputs | | | Outputs | | | |
|---|---|---|---|---|---|---|
| EN | A | B | $Y_3$ | $Y_2$ | $Y_1$ | $Y_0$ |
| 0 | × | × | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 0 | 0 | 1 | 0 |
| 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 |

## Schematic: -



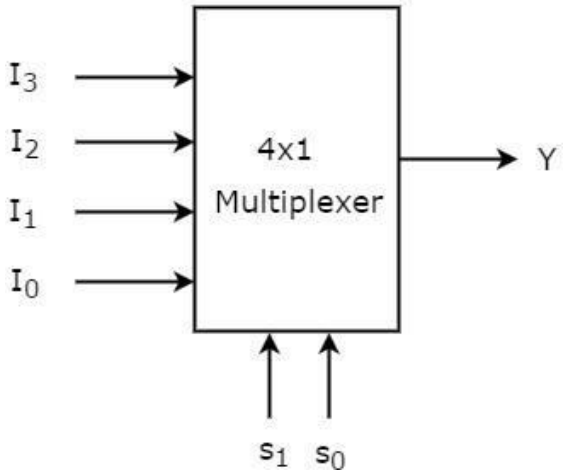
## Multiplexers: -

Multiplexer is a combinational circuit that has maximum of 2n data inputs, 'n' selection lines and single output line. One of these data inputs will be connected to the output based on the values of selection lines.

Since there are 'n' selection lines, there will be 2n possible combinations of zeros and ones. So, each combination will select only one data input. Multiplexer is also called as Mux.

## Block Diagram: -

4x1 Multiplexer has four data inputs $I_3$, $I_2$, $I_1$ & $I_0$, two selection lines $s_1$ & $s_0$ and one output Y. The block diagram of 4x1 Multiplexer is shown in the following figure: -




## Truth Table: -

One of these 4 inputs will be connected to the output based on the combination of inputs present at these two selection lines. Truth table of 4x1 Multiplexer is shown below.

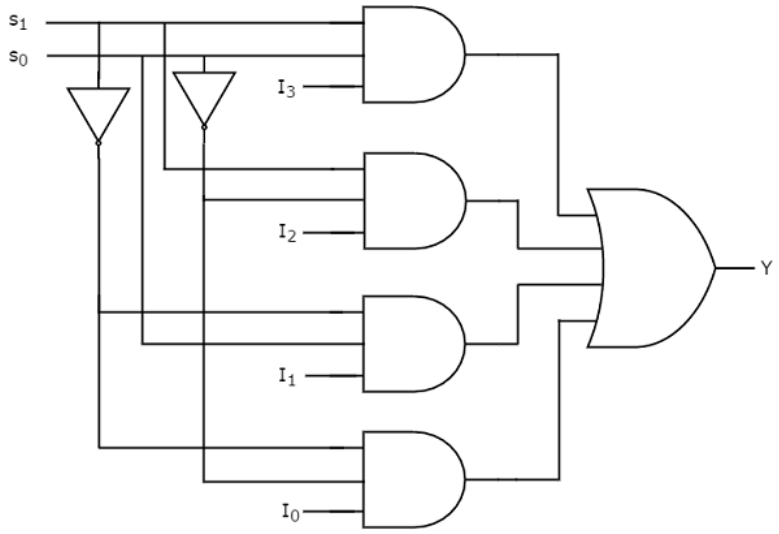
| Selection Lines | | Output |
|---|---|---|
| $S_1$ | $S_0$ | Y |
| 0 | 0 | $I_0$ |
| 0 | 1 | $I_1$ |
| 1 | 0 | $I_2$ |
| 1 | 1 | $I_3$ |

## Schematic Diagram: -

From Truth table, we can directly write the Boolean function for output, Y as

$$Y=S1'S0'I0+S1'S0I1+S1S0'I2+S1S0I3$$

We can implement this Boolean function using Inverters, AND gates & OR gate. The circuit diagram of 4x1 multiplexer is shown in the following figure.



# 3. Tasks

1. Write a Gate Level Verilog and Behavioral level Code for Half Adder and Full Adder. Execute the code on ZYBO and verify results using different instances. Analyze time & size estimations. Draw Graphs to show differences.

2. Write a Verilog Code for 2x4 decoder. Implement the code on ZYBO and verify results using different instances. Analyze time & size estimations. Draw Graphs to show differences.

3. Write a Verilog Code for 4x1 Multiplexer. Implement the code on ZYBO and verify results using different instances. Analyze time & size estimations. Draw Graphs to show differences.